

Exam-like Problems Project-03

Q1

- 1) Caller-saved registers that preserved: a0, a1, t0, t1
- 2) Callee-saved registers that preserved: None
- 3) Are there caller saved registers used/not preserved? Yes, t2, t3, t4, t5, t6.
This is ok because there's no function calls for these registers and there's only a small scope within that function.
- 4) How many bytes used in function by the stack? 40 bytes
- 5) Pointer-based array or indexed array access? Pointer-based

Q2

This function checks all amount of the letter 'c' in a string and prints it using recursion and adding addval to the recursive call which will return the total amount. In Riscv:

.global count-rec-s

count-rec-s:

li t0, 0(a0)

beq t0, zero, done # if str[0] == '\0'

li t1, 0 # addval is 1

beq t0, a1, add # if str[0] == 'c', go to add

addi a0, a0, 1

call count-rec-s

add a0, a0, t1

ret

add:

li t1, 1 # addval = 1

addi a0, a0, 1

call count-rec-s

add a0, a0, t1

ret

done:

li a0, 0

ret

Q3

- 1) idcz is 2 bytes, int val is 4 bytes, the struct is 8 bytes, & padding is 2 bytes between the char and int. So, 16 actual bytes. If there was more alignment like with 32 or 64-byte alignment, there's more padding bytes, but not actual bytes which is 16.
- 2) To load the next-p field value into t0 you would need to use offset, so ld t0, 8(a0). & because you're getting the next-p location.